

PDF to Audiobook

Brief: What It Is and How It Works

Explainer document generated from source

What it does

PDF to Audiobook is a Python desktop tool: drag a batch of PDFs onto the window (or pick them with a file dialog), optionally preview extracted text and restrict the page range, choose a language, accent, and speech rate, click Convert All, and get back one MP3 per PDF with a per-file and overall progress bar along the way. A Play button plays the result straight from the window. It also runs from the command line with no window at all. The whole project is still two files, deliberately kept separate: a conversion engine and a GUI shell around it, the latter now running conversion on a background thread.

Terms used below

GUI A windows-and-buttons interface, built here with **Tkinter** plus the optional **tkinterdnd2** package for drag-and-drop.

TTS Text-to-Speech: converting written text to spoken audio. Done here by **gTTS**, a client for Google Translate's public speech-synthesis web service (requires internet, no API key, no volume control).

PDF text extraction

Reading the words back out of a PDF, done here by **PyPDF2**. PDFs are a visual format, so extraction is not always perfect (scanned/image-only pages yield no text).

Worker thread

A second line of execution that runs the slow network-bound conversion work so the window's own thread stays free to redraw and respond to clicks, coordinated through a thread-safe `queue.Queue`.

Architecture: two modules, one seam, one queue

- `converter.py` holds all the real work and has no GUI dependency, so it can run headless and be tested on its own:
 - `get_page_count` and `extract_text_from_pdf/extract_pages_from_pdf` read a PDF, optionally restricted to a 1-based, inclusive page range, clamped safely if the requested range is out of bounds.
 - `convert_pdf_to_audio(...)` now synthesizes speech **one page at a time** instead of the whole document in one call, so it can invoke an optional `progress_callback(done,`

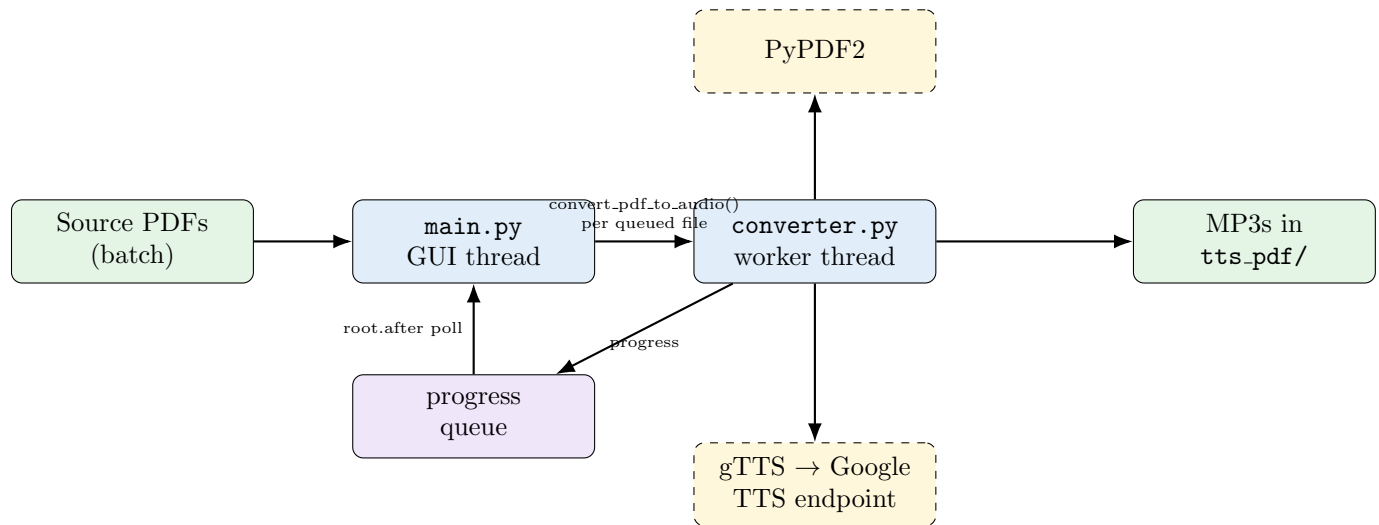


Figure 1: `main.py` never touches `PyPDF2` or `gTTS` directly. It only calls `converter.py`’s public functions, and the only thing that crosses back from the worker thread is a small message on a thread-safe queue.

`total`) after every page. The resulting per-page mp3 fragments are written one after another into the same file (valid because mp3 frames are self-contained), first to a `.part` temp file, then atomically renamed into place with `os.replace` once conversion succeeds, so a failed run never leaves a broken file at the expected output path. It also takes `lang`, `tld` (accent), and `slow` (gTTS’s own rate control) parameters.

- `find_audio_player/play_audio` look for `ffplay`, `mpv`, `afplay`, or `aplay` on the system `PATH` with `shutil.which` and run whichever is found first; if none exist, `play_audio` returns `False` instead of failing silently.
- A full `argparse` CLI exposes every option above headless, including `--play`.
- `main.py` builds the window: a `Treewiew` queue table (drag-and-drop enabled if `tkinterdnd2` is installed, Add Files always available), a Preview / Page Range dialog, a voice-options panel, and a Convert All button with an overall progress bar.
 - Each queued PDF is a `QueueItem`: its path, page count, chosen page range, status text, and output path once converted.
 - `start_conversion` runs the batch on a `threading.Thread`. The worker puts small tuples like `("page_progress", index, done, total)` onto a `queue.Queue` instead of touching any widget directly (unsafe off the main thread); the main thread drains that queue every 100ms via `root.after` and updates the table and progress bar from there.
 - Play Selected Output runs `play_audio` on its own short daemon thread too, since playback blocks until the player exits, and shows a clear “no player found” message with the file path if none of the four known players are installed.

How the pieces talk to each other

The GUI/converter boundary is still exactly one function call wide: `main.py` calls `converter.py`'s `convert_pdf_to_audio`, once per queued file, now from a background thread instead of the main one. The only addition is the progress queue, which exists purely to get status updates back across that thread boundary safely; it does not change what either module is responsible for. Unlike the previous version, the window **stays responsive** during conversion: it can be moved, resized, and its buttons still register while a multi-file batch is in progress, because the network calls happen entirely off the main thread.

A full conversion, in one picture

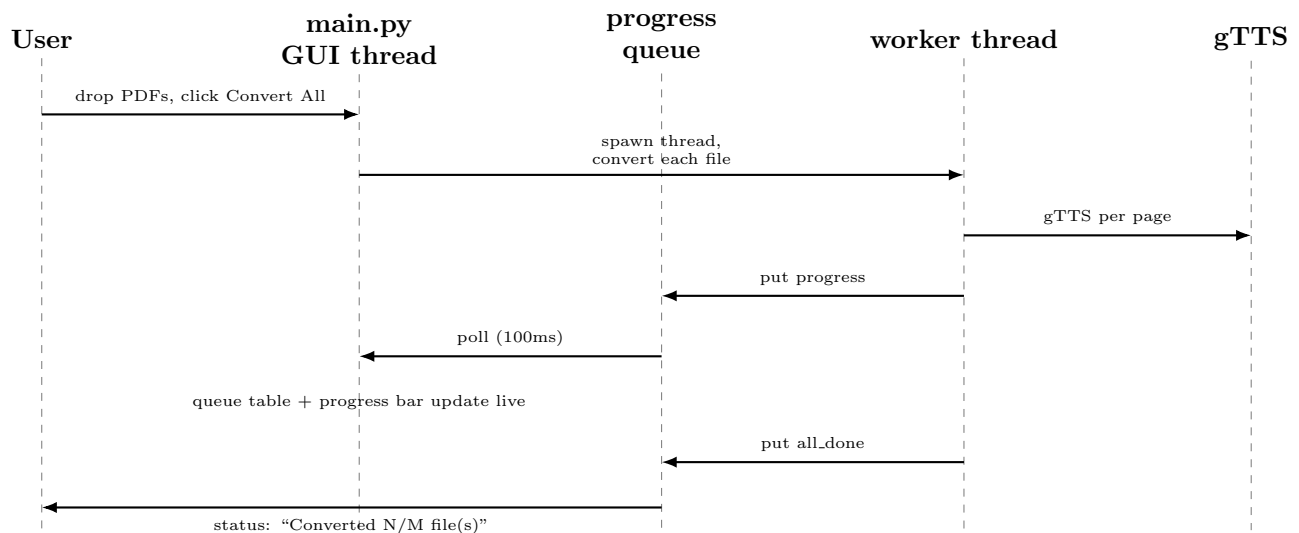


Figure 2: Every arrow through the progress queue crosses a thread boundary; nothing else does. The GUI thread only ever blocks on cheap queue reads, never on network calls.

Output

Each converted PDF produces one MP3, named `<original_name>.tts.mp3`, saved in a `tts.pdf` folder created next to the source PDF (or a custom folder passed to the CLI's `-o` flag), for every file in the batch.

Dependencies

Three pinned third-party packages: `PyPDF2==3.0.1` (text extraction), `gTTS==2.5.4` (speech synthesis, requires internet, no API key), and `tkinterdnd2==0.6.2` (drag-and-drop, optional at runtime). Tkinter, `threading`, `queue`, `subprocess`, `shutil`, `argparse`, and `os` come from the standard library. Local playback needs an external player binary on `PATH`; it is not a Python dependency and is checked for, not assumed, at runtime.

Known limitations (stated in the project itself)

- **No offline fallback.** gTTS needs internet access; there is no local TTS engine as a backup.
- **No volume control.** gTTS does not expose one, so none is offered in the UI rather than faking it.
- **Playback needs an external program.** If none of `ffplay`, `mpv`, `afplay`, or `aplay` is installed, the app cannot play audio itself and says so plainly instead of failing silently.
- **mp3 concatenation is informal.** Per-page mp3 fragments are joined by raw byte concatenation rather than a proper audio-muxing library; it decoded correctly in every player tested, but it is relying on mp3's frame independence rather than a documented container-level guarantee.

Summary

The project's central design decision, a clean split between a GUI-free conversion module and a thin GUI shell, survives this revision intact and is what made every new feature (page-by-page progress, page ranges, accents, playback, a background thread) addable to one side or the other without entangling the two. The one addition to the seam itself is the progress queue, which exists solely to move status updates safely across a thread boundary the previous single-threaded version never had to cross.